

Project Report for Snakes and Ladders Game

Group Members: Anurag Pawar(B24MT1007)

Arpit(B24CS1015)

T. Manisha Beniwal(B24PH1045)

Ayaan Tanweer(B24PH1006)

1. Introduction:

The **Snakes and Ladders Game** is a simulation of the classic board game where two players take turns rolling a die to move their pieces across a board. The game has snakes and ladders placed at specific positions, which either bring players down (snakes) or move them up (ladders). The goal is to be the first player to reach square 100. The project was implemented using **C** and the **GTK** graphical user interface (GUI) library to provide a simple and engaging graphical interface.

2. Design & Architecture:

The game was designed to be modular and efficient. The design is based on the following key components:

- ⇒ **Game Board:** The board consists of 100 squares, with snakes and ladders placed at predefined positions.
- ⇒ **Player Movement:** The players' movements are based on the die rolls. When a player rolls the die, their piece moves forward, and if they land on a snake or ladder, they are moved accordingly.
- ⇒ **Score Management:** The game tracks the score of each player in a file called score.txt, which is updated whenever a player wins a game.
- ⇒ **Graphical User Interface (GUI):** The interface was created using GTK, where the board image and player images are displayed on a window. Players can click a button to roll the die, and the resulting die roll is shown on the screen.

3. Implementation:

The game is implemented with the following key features:

1. **Random Die Roll:** The die roll is simulated using the rand() function in C, which generates a random number between 1 and 6.

```
int rollDie() {  
    return rand() % 6 + 1;  
}
```

2. **Player Movement:** The movement of players on the board is handled using a simple formula. Players move by the number rolled, and if they land on a snake or ladder, the corresponding adjustment is made to their position.

```
int movePlayer(int currentPlayer, int roll) {
    int newPosition = currentPlayer + roll;
    return newPosition + snakesAndLadders[newPosition];
}
```

3. **Snakes and Ladders Logic:** A 101-size array holds the snake and ladder positions. Snakes bring players backward, and ladders push players forward.

```
int snakesAndLadders[101] = {0};
snakesAndLadders[16] = -13; // Snake at 16 moves to 3
snakesAndLadders[5] = 4; // Ladder at 5 moves to 9
```

4. **Graphical Interface:** The `gtk_fixed_move` function moves the player images on the board, based on their updated positions:

```
void movePlayerImage(GtkWidget *player_image, int position) {
    ...
    gtk_fixed_move(GTK_FIXED(fixed), player_image, col * 60, row * 60);
}
```

5. **Score Tracking:** The scores are read from and written to a text file called `score.txt`. When a player wins, the score is updated:

c

Copy code

```
void updateScore(int player) {
    ...
    fprintf(file, "p1-%d\np2-%d", p1, p2);
}
```

4. Results:

=> **Gameplay:** The game runs smoothly with two players rolling the die and moving across the board.

=> **Winning Condition:** The first player to reach square 100 wins the game. Once a player wins, their score is saved in the `score.txt` file.

- **Score Management:** The score file correctly reflects the number of wins for each player after every game.

5. Conclusion:

The **Snakes and Ladders Game** provides a fun and interactive graphical interface for a classic board game. By using GTK and C, the game incorporates randomness in player movement and integrates file handling for score tracking. The project is a success, and further improvements could include adding sound effects, animations, and multiplayer capabilities over a network.

6. Future Work:

=> **Multiplayer Support:** Implementing an online multiplayer feature so players can play over a network.

=> **Enhanced Graphics:** Using animations to enhance the movement of player pieces and improve the visual experience.

=> **Game Difficulty Levels:** Introducing different levels of difficulty by varying the number of snakes and ladders or adding more complex rules.